Karolina Lęcznar

Python Automation & Process Optimization Engineer

About me

Diversely skilled Engineer with hands-on experience in API integration, application development, database technologies, and AI-powered solutions. Passionate about building practical software tools, automating repetitive tasks, continuously expanding programming skills to build scalable solutions.

Core expertise: Python · REST API Integration · Workflow Automation · SQL · Power Automate · Data Processing

Portfolio samples

SHELL FAIR API Automation

Production Python REST API

Business problem

Creating Shell FAIRs required repetitive manual work and user interaction with the Net-Inspect system.

Solution

Developed a Python automation that retrieves FAIR data, identifies records requiring a Shell FAIR, creates the required records and updates data through REST API endpoints.

Technical implementation

- GET, POST and PUT requests
- API response and error handling
- Rate limiting, pagination
- Refactored from previously built browser-based automation to direct API integration

Business impact

Improved reliability and reduced dependence on UI changes compared with the original browser-based approach. **30% reduction in FAIR-related HelpDesk tickets.**

Net Inspect API Authentication & Connection

```python
def build_hmac_signature(app_id, secret_key, http_method, absolute_uri, body=""):
    nonce = uuid.uuid4().hex
    timestamp = str(int(time.time()))

    # Base64 MD5 hash of body, or empty string if no body
    if body:
        md5_hash = hashlib.md5(body.encode("utf-8")).digest()
        content_base64 = base64.b64encode(md5_hash).decode()
    else:
        content_base64 = ""  # IMPORTANT: empty string per API doc

    # Lowercase full absolute URI
    absolute_uri = absolute_uri.lower()

    # Raw signature string
    raw_signature = f"{app_id}\n{http_method}\n{absolute_uri}\n{timestamp}\n{nonce}\n{content_base64}\n"

    # Compute HMAC SHA256
    decoded_secret = base64.b64decode(secret_key)
    hmac_digest = hmac.new(decoded_secret, raw_signature.encode("utf-8"), hashlib.sha256).digest()
    hmac_signature = base64.b64encode(hmac_digest).decode()

    return f"{app_id}.{hmac_signature}.{nonce}.{timestamp}"
```

```python
def get_auth_token():

    credentials = {
        "userId": USER_ID,
        "password": PASSWORD,
        "company": COMPANY_NAME
    }

    json_body = json.dumps(credentials)

    uri = f"{BASE_URL}/public/{API_VERSION}/token"
    sig = build_hmac_signature(APP_ID, API_SECRET_KEY, "POST", uri, json_body)
    headers = {
        "Authorization": f"Public {sig}",
        "Content-Type": "application/json",
        "Accept": "application/json"
    }

    response = requests.post(uri, headers=headers, data=json_body)

    if response.status_code in (200, 201):
        token = response.json().get("token")
        print("[✓] Retrieved token.\n")
        return token
    else:
        print("[X] Failed to retrieve token.\n")
        print("Status:", response.status_code)
        print(response.text)
        return None
```

Shell FAIR creation

```python
def build_fair_payload(fair_json):
    return {
        "organization": {
            "name": "INCORA",
        },
        "customer": {
            "name": "[redacted]",
        },
        "fromDivision": {
            "name": "[redacted]",
        },
        "toDivision": {
            "name": "[redacted]",
        },
        "program": {
            "name": "[redacted]",  # [redacted]
            },
        "formOne": {
            "standard": "C",
            "parts": {
              "items": [
                {
                  "partNo": fair_json.get("partNo"),
                  "partName": fair_json.get("partName"),
                  "partType": "Detail" if fair_json.get("isDetail") else "Assembly",
                  "supplierName": fair_json.get("organizationName"),
                  "fairNo": fair_json.get("fairNo"),
```

```python
def create_fair(auth_token, payload):
    """
    Create a FAIR in Net-Inspect.
    """
    endpoint = f"/public/{API_VERSION}/firstarticles"
    url = f"{BASE_URL}{endpoint}"  # Full absolute URL for HMAC

    # Prepare JSON body
    body = json.dumps(payload, separators=(",", ":"), sort_keys=True)

    # Build HMAC signature with full URL and current timestamp
    sig = build_hmac_signature(
        APP_ID,
        API_SECRET_KEY,
        "POST",
        url,   # <-- full absolute URL required
        body
    )

    # Build authorization header
    headers = {
        "Authorization": f"Public {sig}:{auth_token}",
        "Accept": "application/json",
        "Content-Type": "application/json",
        "Cache-Control": "no-cache"
    }

    # Send POST request
    response = requests.post(url, headers=headers, data=body)
    # print(response.text)

    # Raise exception if 4xx/5xx
    response.raise_for_status()

    # Return the JSON response
    return response.json()
```

HTTP Errors handling

Any errors had to be fixed manually on the customer side, such as setting up the program, changing the FAIR division, etc.

Any attempts to raise an amendment by the API user credentials were rejected by the customer.

```
except requests.HTTPError as e:
    try:
        error_json = e.response.json()

        validation_errors = (
            error_json
            .get("_responseContext", {})
            .get("validation", {})
            .get("errors", [])
        )

        if any("PROGRAM IS INVALID" in err for err in validation_errors):
            print("")
            print("PART SETUP ON CUSTOMER SIDE IS INVALID FOR PN", partNo)
            print("")
        else:
            print("UNHANDLED API ERROR:", validation_errors)
            print("")

    except Exception:
        print("COULD NOT PARSE ERROR RESPONSE:", str(e))
        print("")
# print(json.dumps(fair_json, indent=2))
```

Supplier Certificate Verification Automation

Production Python Web Scraping Web Automation Data Processing

Business Problem

The Quality department manually verified suppliers' AS/ISO certifications on the OASIS website, downloaded certificates, and prepared reports on a monthly basis.

Solution

Developed a Python automation application to verify supplier certifications on OASIS, download certificates to a designated directory, and automatically generate reports.

Technical Implementation

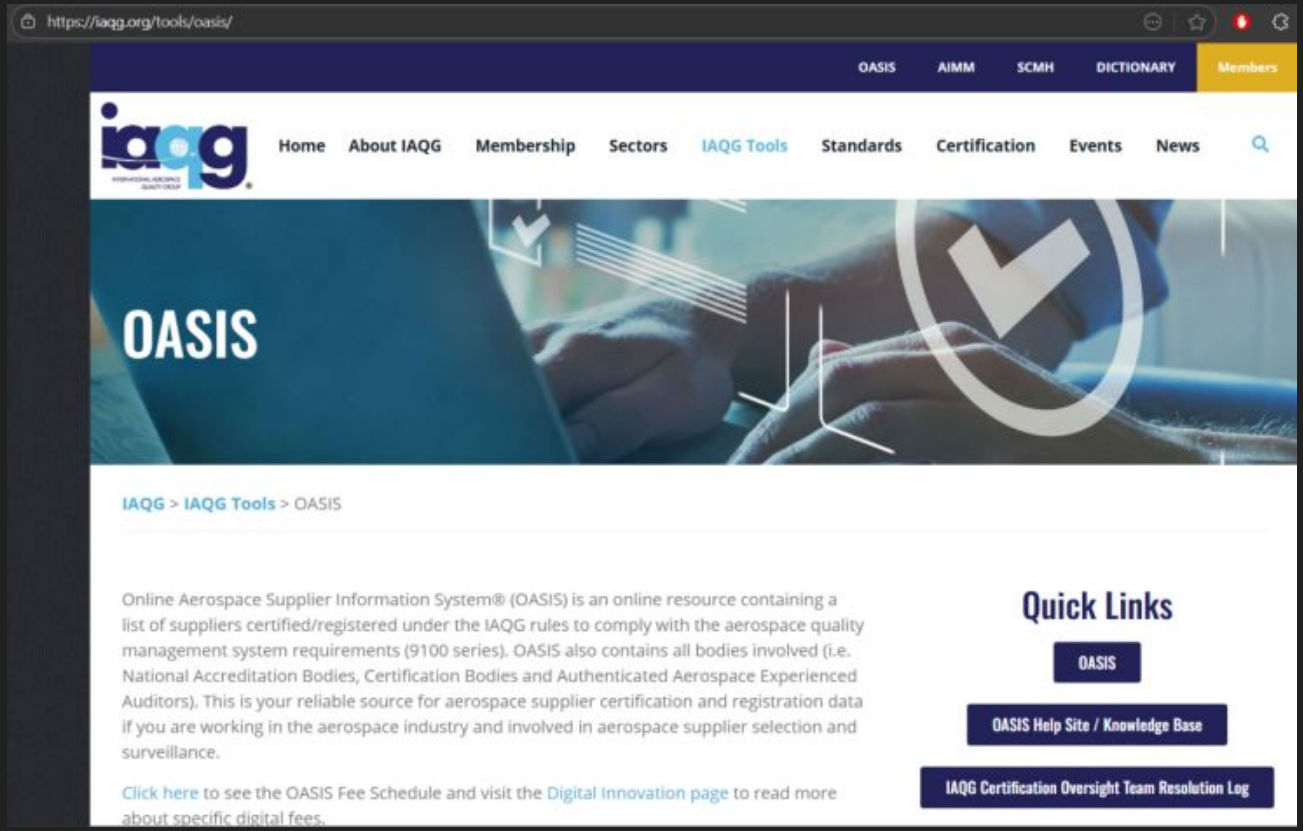
- Automated interactions with the OASIS website to verify supplier AS/ISO certificates.
- Downloaded and organized certificates in a designated directory.
- Generated reports to support monthly certificate verification and tracking.
- Used web automation instead of API integration as OASIS did not provide the API documentation.

Business Impact

Saved approximately 540 hours annually by automating a repetitive process previously performed manually by the Quality department.

Execution Frequency: Monthly.

Utilized OASIS website



Python Automation Code

1. Input Data Collection
   - Retrieve the OIN (OASIS Identification Number) values requiring verification.

2. Web Automation & Report Processing
   - Launch web automation and iterate through OIN numbers.
   - Download the Supplier AS/ISO certificates from the website.
   - Rename each certificate according to the required naming convention.
   - Save the reports to the designated folder on the shared company server.

3. Data Processing & Excel Export
   - Process the collected results using Pandas DataFrames.
   - Export the processed data to an Excel file.
   - Organize and move the generated reports to their designated folders on the shared company server.

```
# get supplier data from website
# download certificate and create report
supplier_name = WebDriverWait(driver, 5).until(EC.presence_of_element_located((
                    By.XPATH, '/html/body/intact-root/div/intact-supplier-detail/div/mat-card/div/d
                ))).text
supplier_address = WebDriverWait(driver, 5).until(EC.presence_of_element_located((
                    By.XPATH, '/html/body/intact-root/div/intact-supplier-detail/div/mat-card/div/d
                ))).text
contact_person = WebDriverWait(driver, 5).until(EC.presence_of_element_located((
                    By.XPATH, '/html/body/intact-root/div/intact-supplier-detail/div/mat-card/div/d
                ))).text
email = WebDriverWait(driver, 5).until(EC.presence_of_element_located((
                    By.XPATH, '/html/body/intact-root/div/intact-supplier-detail/div/mat-card/div/d
                ))).text
cert_status = WebDriverWait(driver, 5).until(EC.presence_of_element_located((
                    By.XPATH, '//*[@id="mat-tab-content-0-0"]/div/div/intact-certificate-details/di
                ))).text
```

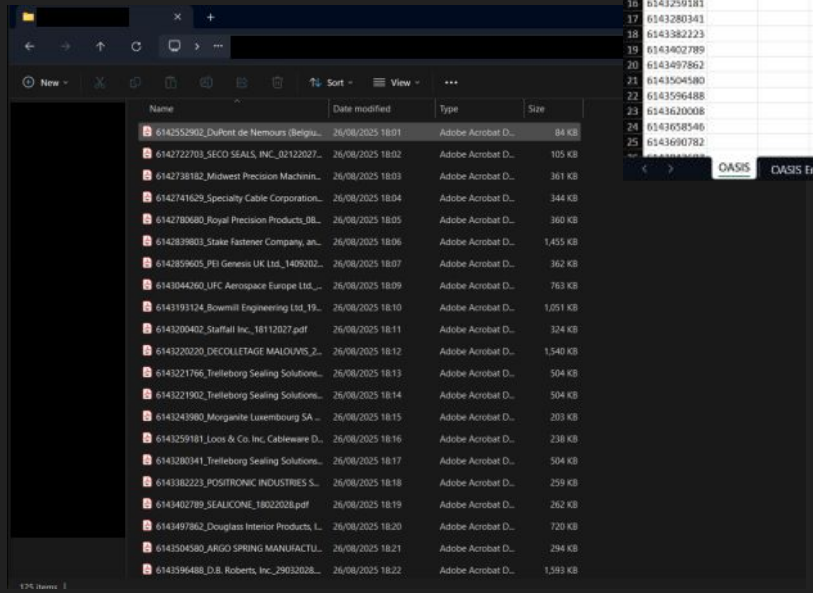
```
# download certificate
download_button = WebDriverWait(driver, 5).until(EC.element_to_be_clickable((
                      By.CSS_SELECTOR, '#mat-tab-content-0-0 > div > div > intact-certificate-de
                  )))
download_button.click()
```

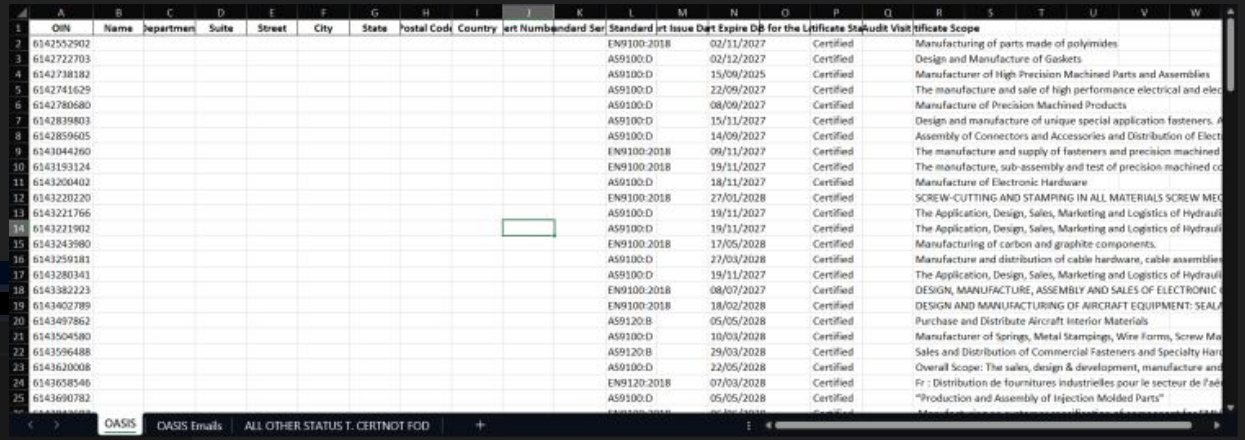
```
def create_report(destination_dir_path):


    # SAVE SCRAPED DATA TO EXCEL

    df1 = pd.DataFrame(
        {
            'OIN' : OIN_found_list,
            'Name': None,
            'Department': None,
            'Suite': None,
            'Street': None,
            'City': None,
            'State': None,
            'Postal Code': None,
            'Country': None,
            'Cert Number': None,
            'Standard Series': None,
            'Standard': standard_results2,
            'Cert Issue Date': None,
            'Cert Expire Date': expiry_results2,
            'Issuing CB for the Latest Cert': None,
            'Certificate Status': status_results2,
            'Last Audit Visit Date': None,
            'Certificate Scope': scope_results2
        }
    )
```

Downloaded AS/ISO Supplier certificates





OIN	Name	Standard	Issue Dat	Expire Da	Certificate Sta	Scope
6142552902		EN9100:2018	02/11/2027		Certified	Manufacturing of parts made of polyimides
6142722703		AS9100:D	02/12/2027		Certified	Design and Manufacture of Gaskets
6142738182		AS9100:D	15/09/2025		Certified	Manufacturer of High Precision Machined Parts and Assemblies
6142741629		AS9100:D	22/09/2027		Certified	The manufacture and sale of high performance electrical and elec
6142780680		AS9100:D	08/09/2027		Certified	Manufacture of Precision Machined Products
6142839803		AS9100:D	15/11/2027		Certified	Design and manufacture of unique special application fasteners. A
6142859605		AS9100:D	14/09/2027		Certified	Assembly of Connectors and Accessories and Distribution of Elect
6143044260		EN9100:2018	09/11/2027		Certified	The manufacture and supply of fasteners and precision machined
6143193124		EN9100:2018	19/11/2027		Certified	The manufacture, sub-assembly and test of precision machined co
6143200402		AS9100:D	18/11/2027		Certified	Manufacture of Electronic Hardware
6143220220		EN9100:2018	27/01/2028		Certified	SCREW-CUTTING AND STAMPING IN ALL MATERIALS SCREW MEC
6143221766		AS9100:D	19/11/2027		Certified	The Application, Design, Sales, Marketing and Logistics of Hydrauli
6143221902		AS9100:D	19/11/2027		Certified	The Application, Design, Sales, Marketing and Logistics of Hydrauli
6143243980		EN9100:2018	17/05/2028		Certified	Manufacturing of carbon and graphite components.
6143259181		AS9100:D	27/03/2028		Certified	Manufacture and distribution of cable hardware, cable assemblies
6143280341		AS9100:D	19/11/2027		Certified	The Application, Design, Sales, Marketing and Logistics of Hydrauli
6143382223		EN9100:2018	08/07/2027		Certified	DESIGN, MANUFACTURE, ASSEMBLY AND SALES OF ELECTRONIC
6143402789		EN9100:2018	18/02/2028		Certified	DESIGN AND MANUFACTURING OF AIRCRAFT EQUIPMENT: SEALA
6143497862		AS9120:B	05/05/2028		Certified	Purchase and Distribute Aircraft Interior Materials
6143504580		AS9100:D	10/03/2028		Certified	Manufacturer of Springs, Metal Stampings, Wire Forms, Screw Ma
6143596488		AS9120:B	29/03/2028		Certified	Sales and Distribution of Commercial Fasteners and Specialty Hard
6143620008		AS9100:D	22/05/2028		Certified	Overall Scope: The sales, design & development, manufacture and
6143658546		EN9120:2018	07/03/2028		Certified	Fr : Distribution de fournitures industrielles pour le secteur de l'aé
6143690782		AS9100:D	05/05/2028		Certified	"Production and Assembly of Injection Molded Parts"

VAT Taxpayer Verification Automation

Production Python Web Scraping Web Automation Data Processing

Business Problem

Accounts Payable (AP) clerks manually verified companies' VAT taxpayer status before releasing payments.

Solution

Developed a Python automation tool to verify companies' active VAT taxpayer status through the Polish Ministry of Finance's VAT Taxpayer Register and automatically save confirmation reports to designated folders.

Technical Implementation

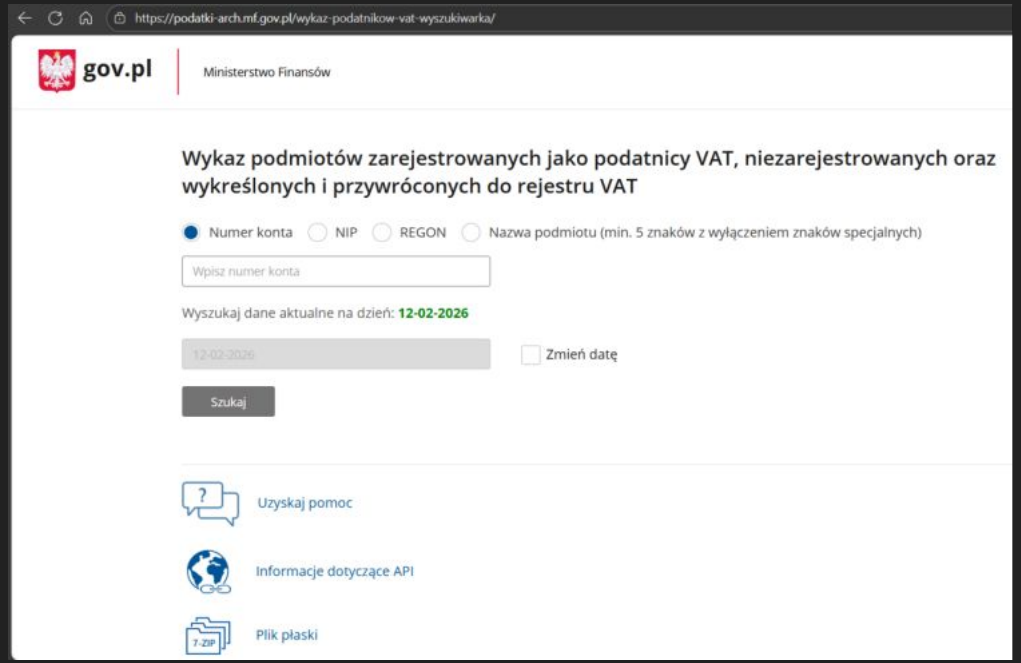
- Automated VAT status verification against the official register.
- Used web scraping because, although REST API integration could support status verification, it did not provide the required functionality to download confirmation records.
- Retrieved and saved confirmation reports for recordkeeping.
- Organized reports into designated folders to support the payment verification process.

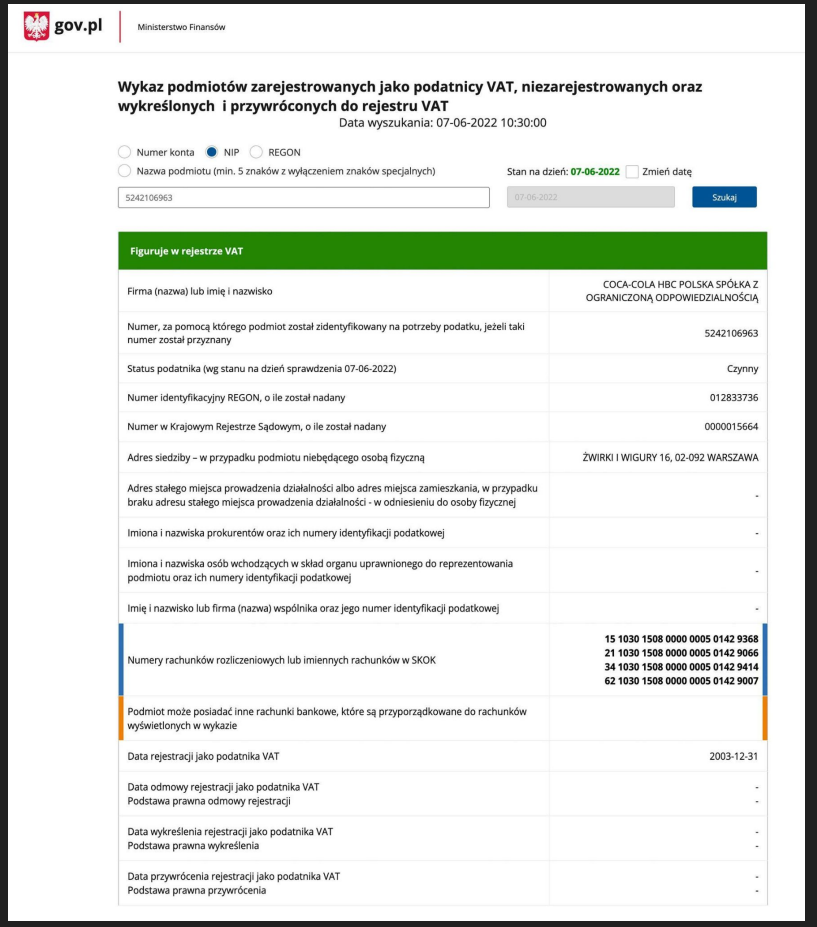
Business Impact

Saved approximately 120 hours annually by eliminating repetitive manual checks.

Scalability

The solution has potential for further development and replication across other Incora divisions.





Python Automation Code

1. Input Data Collection
   - Retrieve the bank account numbers and NIP (Polish Tax Identification Number) values requiring verification.

2. Web Automation & Report Processing
   - Launch web automation and iterate through the bank account numbers and NIP values.
   - Download the VAT confirmation report from the website.
   - Rename each report according to the required naming convention.
   - Save the reports to the designated folder on the shared company server.

3. Data Processing & Excel Export
   - Process the collected results using Pandas DataFrames.
   - Export the processed data to an Excel file.
   - Organize and move the generated reports to their designated folders on the shared company server.

```python
for account in bank_account_list:
    WebDriverWait(driver, 2).until(EC.element_to_be_clickable((
        By.XPATH, '//label[@aria-label="Numer konta"]'
    ))).click()
    WebDriverWait(driver, 2).until(EC.element_to_be_clickable((
        By.XPATH, '//input[@placeholder="Wpisz numer konta"]'
    ))).clear()
    WebDriverWait(driver, 2).until(EC.element_to_be_clickable((
        By.XPATH, '//input[@placeholder="Wpisz numer konta"]'
    ))).click()
    WebDriverWait(driver, 2).until(EC.element_to_be_clickable((
        By.XPATH, '//input[@placeholder="Wpisz numer konta"]'
    ))).send_keys(account)
    time.sleep(2)
    try:
        if elementTwo.is_enabled():
            WebDriverWait(driver, 2).until(EC.element_to_be_clickable((
                By.XPATH, '//button[@aria-label="Szukaj"][@id="sendTwo"]'
            ))).click()
```

```python
# end web scraping
# save data to excel
df_bank = pd.DataFrame(
    {
        'Supplier Code': supplier_code_list_bank,
        'Supplier name': supplier_name_list_bank,
        'Bank Account': bank_account_list,
        'is_registered?':bank_account_list_results
    }
)
df_nip = pd.DataFrame(
    {
        'Supplier Code': supplier_code_list_nip,
        'Supplier name': supplier_name_list_nip,
        'NIP': NIP_list,
        'is_registered?': NIP_list_results
    }
) # create DFs with the lists
```

Net-Inspect KPI Automation

Production Python REST API Pandas SharePoint

Business Problem

The Quality department at the SSC in Poland needed regular KPI reporting across multiple locations, requiring data collection, processing, and manual updates to a shared SharePoint tracker.

Solution

Developed a Python application that retrieves data from the Net-Inspect API, calculates weekly and monthly KPIs for multiple locations, and automatically updates the Quality department's SharePoint tracker.

Technical Implementation

- Integrated with the Net-Inspect REST API to retrieve relevant quality data.
- Processed and aggregated data to calculate weekly and monthly KPIs by location.
- Automated the transfer of calculated KPI results to the designated SharePoint tracker.

Business Impact

Reduced manual reporting effort, improved consistency, and streamlined KPI tracking across multiple locations for the Quality department. **Saved approximately 20 minutes of manual labour every week.**

Net Inspect API Authentication & Connection

```python
def build_hmac_signature(app_id, secret_key, http_method, absolute_uri, body=""):
    nonce = uuid.uuid4().hex
    timestamp = str(int(time.time()))

    # Base64 MD5 hash of body, or empty string if no body
    if body:
        md5_hash = hashlib.md5(body.encode("utf-8")).digest()
        content_base64 = base64.b64encode(md5_hash).decode()
    else:
        content_base64 = ""  # IMPORTANT: empty string per API doc

    # Lowercase full absolute URI
    absolute_uri = absolute_uri.lower()

    # Raw signature string
    raw_signature = f"{app_id}\n{http_method}\n{absolute_uri}\n{timestamp}\n{nonce}\n{content_base64}\n"

    # Compute HMAC SHA256
    decoded_secret = base64.b64decode(secret_key)
    hmac_digest = hmac.new(decoded_secret, raw_signature.encode("utf-8"), hashlib.sha256).digest()
    hmac_signature = base64.b64encode(hmac_digest).decode()

    return f"{app_id}.{hmac_signature}.{nonce}.{timestamp}"
```

```python
def get_auth_token():

    credentials = {
        "userId": USER_ID,
        "password": PASSWORD,
        "company": COMPANY_NAME
    }

    json_body = json.dumps(credentials)

    uri = f"{BASE_URL}/public/{API_VERSION}/token"
    sig = build_hmac_signature(APP_ID, API_SECRET_KEY, "POST", uri, json_body)
    headers = {
        "Authorization": f"Public {sig}",
        "Content-Type": "application/json",
        "Accept": "application/json"
    }

    response = requests.post(uri, headers=headers, data=json_body)

    if response.status_code in (200, 201):
        token = response.json().get("token")
        print("[✓] Retrieved token.\n")
        return token
    else:
        print("[X] Failed to retrieve token.\n")
        print("Status:", response.status_code)
        print(response.text)
        return None
```

Python Automation Code

1. REST API Integration

2. API Data Retrieval for 7 separate divisions
   - Work in Progress (WIP) FAIRs
   - Aged FAIRs
   - Newly Created FAIRs
   - Closed FAIRs

3. WEEKLY KPIs: FAIR Cycle Time Report Download and Processing
   - Update the Sharepoint Tracker with retrieved KPIs

4. MONTHLY KPIs: SLA KPI Calculation
   - Download the FAIR Cycle Time Report from Net Inspect using Web Automation
   - Period Filtering: Select FAIRs closed in the previous calendar month for the EU Hub division
   - SLA Calculation: Calculate the percentage of closed FAIRs completed within 30 business days
   - Update the Sharepoint Tracker

Net Inspect API Authentication & Connection

```python
def build_hmac_signature(app_id, secret_key, http_method, absolute_uri, body=""):
    nonce = uuid.uuid4().hex
    timestamp = str(int(time.time()))

    # Base64 MD5 hash of body, or empty string if no body
    if body:
        md5_hash = hashlib.md5(body.encode("utf-8")).digest()
        content_base64 = base64.b64encode(md5_hash).decode()
    else:
        content_base64 = ""  # IMPORTANT: empty string per API doc

    # Lowercase full absolute URI
    absolute_uri = absolute_uri.lower()

    # Raw signature string
    raw_signature = f"{app_id}\n{http_method}\n{absolute_uri}\n{timestamp}\n{nonce}\n{content_base64}\n"

    # Compute HMAC SHA256
    decoded_secret = base64.b64decode(secret_key)
    hmac_digest = hmac.new(decoded_secret, raw_signature.encode("utf-8"), hashlib.sha256).digest()
    hmac_signature = base64.b64encode(hmac_digest).decode()

    return f"{app_id}.{hmac_signature}.{nonce}.{timestamp}"
```

```python
def get_auth_token():

    credentials = {
        "userId": USER_ID,
        "password": PASSWORD,
        "company": COMPANY_NAME
    }

    json_body = json.dumps(credentials)

    uri = f"{BASE_URL}/public/{API_VERSION}/token"
    sig = build_hmac_signature(APP_ID, API_SECRET_KEY, "POST", uri, json_body)
    headers = {
        "Authorization": f"Public {sig}",
        "Content-Type": "application/json",
        "Accept": "application/json"
    }

    response = requests.post(uri, headers=headers, data=json_body)

    if response.status_code in (200, 201):
        token = response.json().get("token")
        print("[✓] Retrieved token.\n")
        return token
    else:
        print("[X] Failed to retrieve token.\n")
        print("Status:", response.status_code)
        print(response.text)
        return None
```

Weekly & Monthly KPI calculation logic

```
        # Open workbook
        wb = openpyxl.load_workbook(latest_file)
        sheet = wb['Data']

        max_row = sheet.max_row

        # Header row
        sheet.cell(row=1, column=11).value = 30  # K1 threshold
        sheet.cell(row=1, column=12).value = '>< 30 dni'  # L header

        threshold = sheet.cell(row=1, column=11).value  # K1

        # Loop through rows and calculate values
        for row in range(2, max_row + 1):
            # Column L: NETWORKDAYS formula
            sheet.cell(row=row, column=12).value = f"=NETWORKDAYS(B{row},C{row})"

            # Column K: calculate 1/0 based on L and threshold
            l_cell = sheet.cell(row=row, column=12)

            # If L is not yet calculated (formula), we can't rely on its value yet
            # Option 1: calculate using Python directly using dates from B and C
            b_cell = sheet.cell(row=row, column=2).value  # Cycle Start
            c_cell = sheet.cell(row=row, column=3).value  # Cycle End

            if b_cell and c_cell:
                # NETWORKDAYS equivalent in Python
                from numpy import busday_count
                start_date = b_cell.date() if hasattr(b_cell, 'date') else b_cell
                end_date = c_cell.date() if hasattr(c_cell, 'date') else c_cell
                business_days = busday_count(start_date, end_date) + 1  # +1 to match Excel NETWORKDAYS
            else:
                business_days = 0

            k_value = 1 if business_days <= threshold else 0
            sheet.cell(row=row, column=11).value = k_value  # write 1/0 directly

        # Save workbook
        wb.save(latest_file)
        time.sleep(1)
```

```
def calculate_SLA():

    """
    Function calculates SLA KPI
    """

    # Open workbook
    wb = load_workbook(latest_file, data_only=True)

    ws = wb["Data"]

    # Extract all values (including calculated ones)
    data = ws.values
    columns = next(data)
    df = pd.DataFrame(data, columns=columns)

    column_k = pd.to_numeric(df[30], errors='coerce').values

    # --- Ensure column C is datetime ---
    df['Cycle End'] = pd.to_datetime(df['Cycle End'], errors='coerce')

    # --- Determine previous month and correct year ---
    today = datetime.today()
    current_year = today.year
    previous_month = today.month - 1 if today.month > 1 else 12
    previous_month_year = current_year if today.month > 1 else current_year - 1

    # --- Apply filters ---
    filtered_df = df[
        (df['Cycle End'].dt.year == previous_month_year) &
        (df['Cycle End'].dt.month == previous_month) &
        (df['To Division'] == "Pattonair - EU Hub")
    ]

    # --- Count and sum ---
    num_rows = len(filtered_df)
    sum_k = filtered_df.iloc[:, 10].sum()

    SLA_KPI = sum_k / num_rows
    percentage = math.ceil(SLA_KPI * 100)

    print(f"SLA KPI from period: {previous_month_year}-{previous_month:02d}")
    print(f"Number of closed FAIRs: {num_rows}")
    print(f"Number of closed FAIRs within 30 days: {sum_k}")
    print(f"\033[31mSLA KPI: {percentage}%\033[0m\n")

    return percentage, num_rows
```

Tracker Update with Weekly & Monthly KPIs

```python
def tracker_weekly(ftworth_wip, ftworth_aged, ftworth_created, ftworth_closed, euhub_wip, euhub_aged, euhub_created, euhub_closed, lbl_
    """
    Function pastes total numbers of WIP, Submitted, Aged and Closed FAIRs
    for EU-Hub, Fort Worth and LBL division in Weekly sheet
    """
    # Obtain today's week and year
    today = datetime.today()
    week = today.isocalendar().week
    year = today.year

    # Open workbook
    wb = openpyxl.load_workbook(r'Tracker - FAIR.xlsx')
    ws = wb['Tracker - Weekly']

    # Find the last row that actually has a value in column A
    last_data_row = 1
    for row in range(1, ws.max_row + 1):
        if ws.cell(row=row, column=1).value is not None:
            last_data_row = row

    next_row = last_data_row + 1

    # === Prepare rows to insert ===
    rows_to_add = [
        [week, year, "FORT WORTH", safe_len(ftworth_created), safe_len(ftworth_closed), safe_len(ftworth_wip), safe_len(ftworth_aged)],
        [week, year, "EU HUB", safe_len(euhub_created), safe_len(euhub_closed), safe_len(euhub_wip), safe_len(euhub_aged)],
        [week, year, "LBL", safe_len(lbl_created), safe_len(lbl_closed), safe_len(lbl_wip), safe_len(lbl_aged)],
    ]

    # === Append all rows ===
    for i, row_data in enumerate(rows_to_add):
        for col, value in enumerate(row_data, start=1):
            ws.cell(row=next_row + i, column=col, value=value)

    # Save changes
    wb.save(r'Tracker - FAIR.xlsx')
```

```python
def tracker_monthly(euhub_sla, lbl_sla, ftworth_sla):
    """
    Function pastes SLA KPIs for EU-Hub, Fort Worth and LBL division in Monthly sheet
    """

    # Change working directory to the script's location
    os.chdir(os.path.dirname(os.path.abspath(__file__)))

    # Obtain today's month and year
    today = datetime.today()
    month = today.month
    year = today.year

    # Open workbook
    wb = openpyxl.load_workbook(r'Tracker - FAIR.xlsx')
    ws = wb['Tracker - Monthly']

    # Find the last row that actually has a value in column A
    last_data_row = 1
    for row in range(1, ws.max_row + 1):
        if ws.cell(row=row, column=1).value is not None:
            last_data_row = row

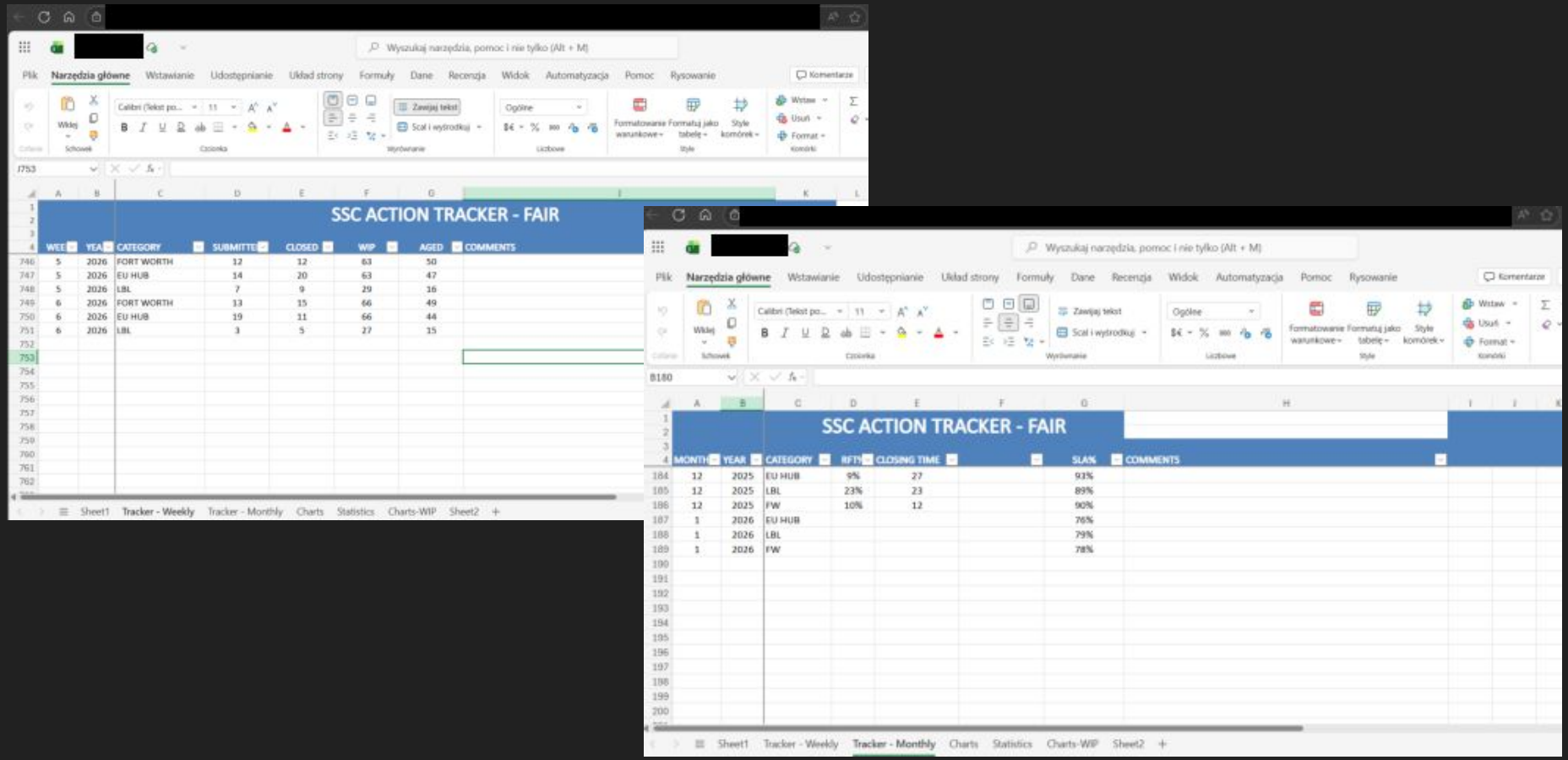
    next_row = last_data_row + 1

    # === Prepare rows to insert ===
    rows_to_add = [
        [month, year, "EU HUB", None, None, None, f"{euhub_sla}%"],
        [month, year, "LBL", None, None, None, f"{lbl_sla}%"],
        [month, year, "FW", None, None, None, f"{ftworth_sla}%"],
    ]

    # === Append all rows ===
    for i, row_data in enumerate(rows_to_add):
        for col, value in enumerate(row_data, start=1):
            ws.cell(row=next_row + i, column=col, value=value)

    # Save changes
    wb.save(r'Tracker - FAIR.xlsx')
```

Final SharePoint KPI tracker



Automated Supplier Follow-Up for Disapproved FAIRs

Python REST API HMAC-SHA256 Pandas Excel Power Automate

Business Problem

Disapproved FAIRs required follow-up with suppliers, particularly when no corrective action had been taken for more than seven days. Manual monitoring and chasing suppliers added to the Quality department's workload.

Solution

Developed a Python script that queries the Net-Inspect REST API, identifies cases with no supplier action for over seven days, and triggers a Power Automate flow to send follow-up reminders.

Technical Implementation

- Integrated with the Net-Inspect REST API to retrieve disapproved FAIR records.
- Applied business rules to identify FAIRs requiring follow-up based on supplier inactivity exceeding 7 days.
- Connected the Python process with Power Automate to automate supplier reminder notifications.

Business Impact

Reduced manual monitoring and follow-up effort, helping the Quality department maintain timely supplier responses and progress on disapproved FAIRs.

Net Inspect API Authentication & Connection

```python
def build_hmac_signature(app_id, secret_key, http_method, absolute_uri, body=""):
    nonce = uuid.uuid4().hex
    timestamp = str(int(time.time()))

    # Base64 MD5 hash of body, or empty string if no body
    if body:
        md5_hash = hashlib.md5(body.encode("utf-8")).digest()
        content_base64 = base64.b64encode(md5_hash).decode()
    else:
        content_base64 = ""  # IMPORTANT: empty string per API doc

    # Lowercase full absolute URI
    absolute_uri = absolute_uri.lower()

    # Raw signature string
    raw_signature = f"{app_id}\n{http_method}\n{absolute_uri}\n{timestamp}\n{nonce}\n{content_base64}\n"

    # Compute HMAC SHA256
    decoded_secret = base64.b64decode(secret_key)
    hmac_digest = hmac.new(decoded_secret, raw_signature.encode("utf-8"), hashlib.sha256).digest()
    hmac_signature = base64.b64encode(hmac_digest).decode()

    return f"{app_id}.{hmac_signature}.{nonce}.{timestamp}"
```

```python
def get_auth_token():

    credentials = {
        "userId": USER_ID,
        "password": PASSWORD,
        "company": COMPANY_NAME
    }

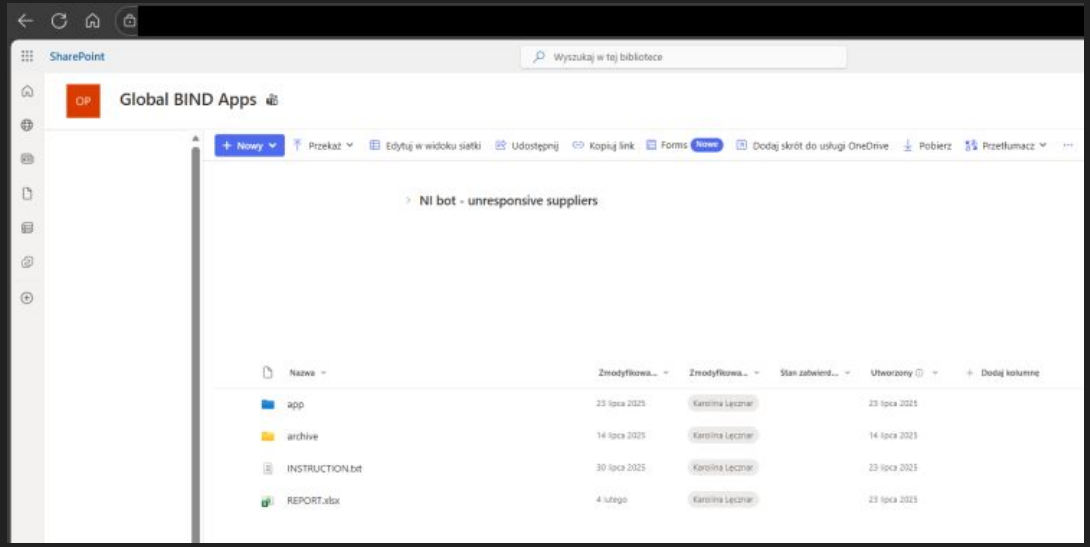
    json_body = json.dumps(credentials)

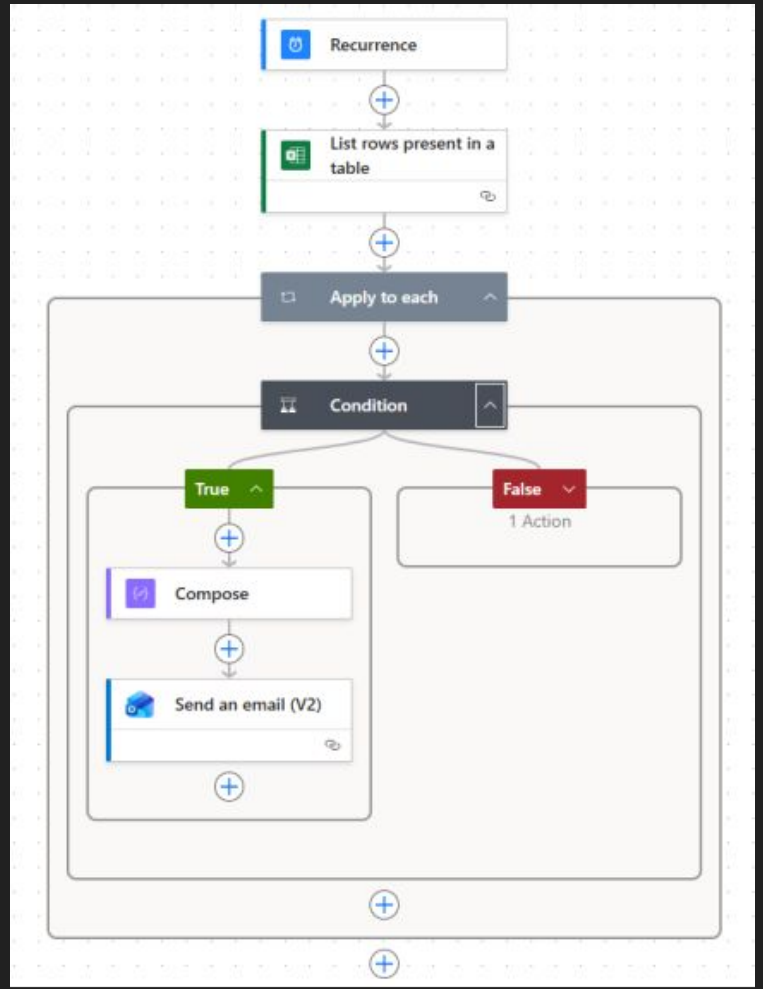
    uri = f"{BASE_URL}/public/{API_VERSION}/token"
    sig = build_hmac_signature(APP_ID, API_SECRET_KEY, "POST", uri, json_body)
    headers = {
        "Authorization": f"Public {sig}",
        "Content-Type": "application/json",
        "Accept": "application/json"
    }

    response = requests.post(uri, headers=headers, data=json_body)

    if response.status_code in (200, 201):
        token = response.json().get("token")
        print("[✓] Retrieved token.\n")
        return token
    else:
        print("[X] Failed to retrieve token.\n")
        print("Status:", response.status_code)
        print(response.text)
        return None
```

```
row = {
    'FAIR ID': article.get('fairNo'),
    'Part Number': article.get('partNo'),
    'Part Name': article.get('partName'),
    'Supplier Name': article.get('organizationName'),
    'Supplier e-mail': article.get('approvalStatusUser') or "",
    'Division': article.get('toDivision', {}).get('name') if article.get('toDivision') else "",
    'Last Update Date': last_update,
    'Number of Iterations': article.get('attempts'),
    'Reviewer (Surname, Name)': reviewer,
    'Days since Last Update': days_since_update
}
rows.append(row)
```





Automated Inspection Error KPI Reporting

Python Pandas Excel Power Automate SharePoint

Business Problem

The Quality department at the SSC in Poland needed to process inspection error reports received via Outlook, calculate performance KPIs, and manually update a centralized SharePoint tracker.

Solution

Developed an automated workflow that transfers Excel reports from Outlook to a designated SharePoint folder using Power Automate, processes the files in Python, calculates KPIs, and updates the SharePoint tracker.

Technical Implementation

- Automated File Handling: Used Power Automate to move incoming XLSX files from Outlook to a designated SharePoint folder.
- Data Processing: Used Python and Pandas to read, clean, and analyze inspection error data.
- KPI Calculation: Automated the calculation of relevant quality performance indicators.
- SharePoint Integration: Transferred the calculated KPI results to the designated SharePoint tracker.

Business Impact

Reduced manual data handling and reporting effort, improved consistency, and streamlined KPI reporting for the Quality department.

Net Inspect API Authentication & Connection

```python
def build_hmac_signature(app_id, secret_key, http_method, absolute_uri, body=""):
    nonce = uuid.uuid4().hex
    timestamp = str(int(time.time()))

    # Base64 MD5 hash of body, or empty string if no body
    if body:
        md5_hash = hashlib.md5(body.encode("utf-8")).digest()
        content_base64 = base64.b64encode(md5_hash).decode()
    else:
        content_base64 = ""  # IMPORTANT: empty string per API doc

    # Lowercase full absolute URI
    absolute_uri = absolute_uri.lower()

    # Raw signature string
    raw_signature = f"{app_id}\n{http_method}\n{absolute_uri}\n{timestamp}\n{nonce}\n{content_base64}\n"

    # Compute HMAC SHA256
    decoded_secret = base64.b64decode(secret_key)
    hmac_digest = hmac.new(decoded_secret, raw_signature.encode("utf-8"), hashlib.sha256).digest()
    hmac_signature = base64.b64encode(hmac_digest).decode()

    return f"{app_id}.{hmac_signature}.{nonce}.{timestamp}"
```

```python
def get_auth_token():

    credentials = {
        "userId": USER_ID,
        "password": PASSWORD,
        "company": COMPANY_NAME
    }

    json_body = json.dumps(credentials)

    uri = f"{BASE_URL}/public/{API_VERSION}/token"
    sig = build_hmac_signature(APP_ID, API_SECRET_KEY, "POST", uri, json_body)
    headers = {
        "Authorization": f"Public {sig}",
        "Content-Type": "application/json",
        "Accept": "application/json"
    }

    response = requests.post(uri, headers=headers, data=json_body)

    if response.status_code in (200, 201):
        token = response.json().get("token")
        print("[✓] Retrieved token.\n")
        return token
    else:
        print("[X] Failed to retrieve token.\n")
        print("Status:", response.status_code)
        print(response.text)
        return None
```

Data processing and KPI calculations

```python
def main():

    INSPECTION_FILE = get_resource_path("Inspection Errors.xlsx")
    wb = openpyxl.load_workbook(INSPECTION_FILE)
    results = {}

    for name in KPI_ORDER:
        cfg = INSPECTIONS[name]

        data, total_value = read_source_file(cfg["file"], cfg["sheet"])

        row_count, fair_count, fair_value = process_target_sheet(
            wb, name, data
        )

        results[name] = {
            "row_count": row_count,
            "total_value": total_value,
            "fair_count": fair_count,
            "fair_value": fair_value,
        }

    write_kpi(wb, results)
    wb.save(INSPECTION_FILE)
```

```python
# ============================================================
# MAIN PIPELINE
# ============================================================

def main():

    INSPECTION_FILE = get_resource_path("Inspection Errors.xlsx")
    wb = openpyxl.load_workbook(INSPECTION_FILE)
    results = {}

    for name in KPI_ORDER:
        cfg = INSPECTIONS[name]

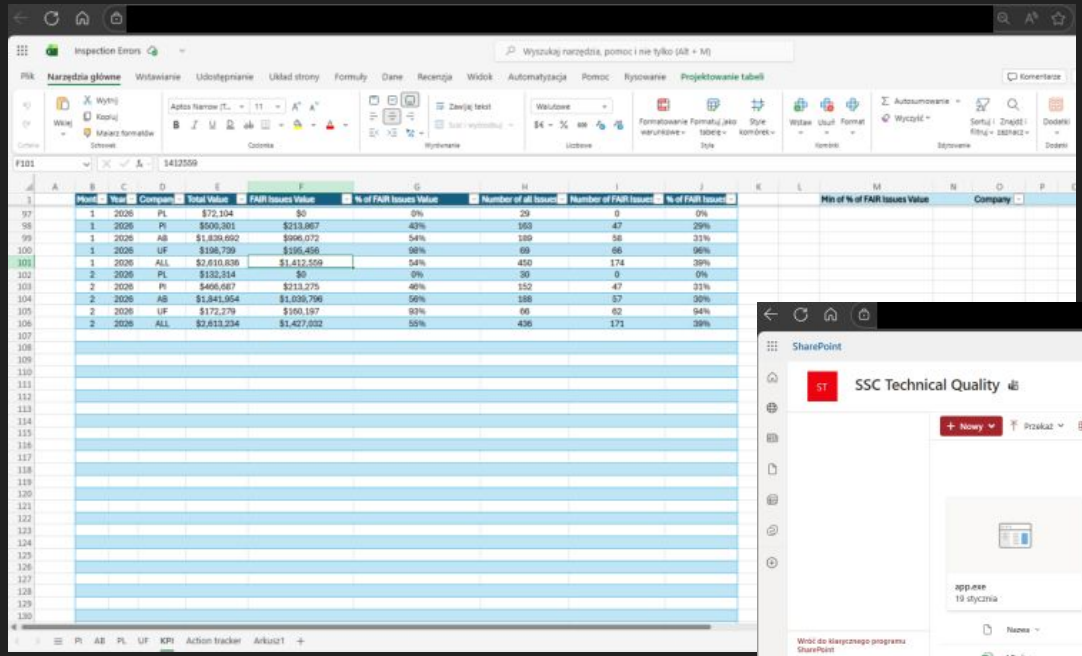
        data, total_value = read_source_file(cfg["file"], cfg["sheet"])

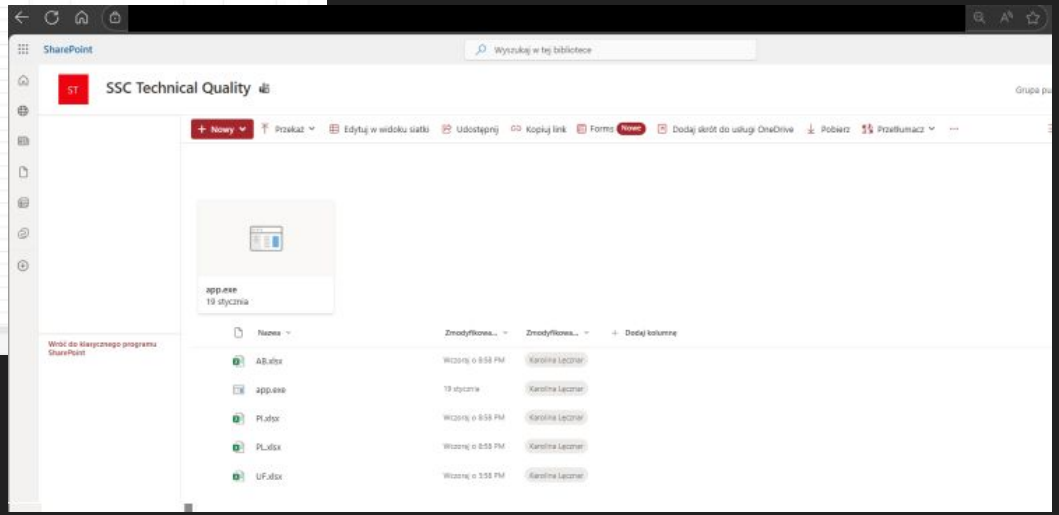
        row_count, fair_count, fair_value = process_target_sheet(
            wb, name, data
        )

        results[name] = {
            "row_count": row_count,
            "total_value": total_value,
            "fair_count": fair_count,
            "fair_value": fair_value,
        }

    write_kpi(wb, results)
    wb.save(INSPECTION_FILE)
```

File transfer to Sharepoint via Power Automate flow + Updated Sharepoint Tracker





FORM 1 HEADER & PRICE REDACTION TOOL

Python PDF Processing PyMuPDF (fitz) pypdf OCR Executable Packaging

Business Problem

The Quality department at the SSC Poland needed to prepare Form 1 documents by adding company data and inspector stamps, while redacting sensitive purchase order details such as prices and lead times.

Solution

Developed a standalone Python executable that automates Form 1 document preparation, applies Incora's header and the inspector's stamp, and redacts designated sensitive information from purchase orders.

Technical Implementation

- Document Customization: Automatically added Incora's company header and the inspector's stamp to FAIR Form 1
- Sensitive Data Redaction: Redacted confidential purchase order data, including prices and lead times.
- PDF Processing: Used Python PDF libraries and OCR to support document manipulation.
- Executable Application: Packaged the tool as an executable for convenient use by the Quality department without requiring manual script execution.

Business Impact

Streamlined document preparation, reduced repetitive manual editing and supported consistent handling of sensitive purchase order information.

BEFORE

AFTER

FAIR FORM 1

Jurisdiction: NO LICENCE REQUIRED | Classification: Not Listed

AS/EN/SJAC9102 Rev B First Article Inspection

Form 1: Part Number Accountability

1. Part Number:	2. Part Name:	3. Serial Number:	4. FAIR Number: 1061 Internal FAIR Number: 00094506
91213164	SCREW, SOCKET HEAD, DRILLED CAP, CRS, 4 (.112) -40 UNC-2A	N/A	
5. Part Revision Level:	6. Drawing Number:	7. Drawing Revision Level:	8. Additional Changes:
2	91213161 TO 91213169	2	N/A
9. Manufacturing Process Reference:	10. Organization Name:	11. Supplier Code:	12. P.O. Number:
W/O 00094506	PRECISION AEROSPACE COMPONENT ENG G LTD. - BRAINTREE	PREC01	9345719 LINE ITEM 1
13. Detail FAI: ☑ Assembly FAI: ☐	14. Full FAI: ☑ Partial FAI: ☐ Baseline Part Number (Including Revision Level) N/A		
	Reason for Partial:		AOG ☐ FAA Approved ☐
a) If above part number is a detail part only, go to Field 19 b) If above part number is an assembly, go to the "INDEX" section below.			
INDEX of part numbers or sub-assembly numbers required to make the assembly noted above.			
15. Part Number	16. Part Name	17. Part Serial Number	Supplier / 18. FAIR Number
N/A	N/A	N/A	N/A / N/A
1) Signature indicates that all characteristics are accounted for; meet drawing requirements or are properly documented for disposition. 2) Also indicate if the FAI is complete per Section 4.4.			
Customer INCORA		Program NO LICENCE REQUIRED	
From Division - To Division - Pattonair - EU Hub			
FAI Complete ◉ FAI Not Complete ○		Pass ◉ Fail ○	
19. Signature James Mullan		20. Date August 5, 2022 12:20:26 PM	
21. Reviewed By Kasia Czech Digitally signed using Net-Inspect		22. Date August 5, 2022 12:21:00 PM	
23. Customer Approval		24. Date	

Jurisdiction: NO LICENCE REQUIRED | Classification: EAR 9A991.D

Jurisdiction: UK DUAL USE | Classification: PL9009A.2.A

PATTONAIR LTD
Vendor #: 128521
PO: WLPAT3475 & Item 00010
PN: CH1541-0095 rev: A
Unique FAIR Id: 123456

AS/EN/SJAC9102 Rev C First Article Inspection

Form 1: Part Number Accountability

1. Part Number:	2. Part Name:	3. Serial Number:	4. FAIR Identifier: 1047
CH1541-0095	BUSH-SPLIT	N/A	
5. Part Revision Level:	6. Drawing Number:	7. Drawing Revision Level:	8. Additional Changes:
A	CH1541-0095	A	N/A
9. Manufacturing Process Reference:	10. Organization Name:	11. Supplier Code:	12. Purchase Order Number:
H1332	ENSINGER MACHINED PARTS - SOMERSET	TRIG01	V390837 LINE 2
13. Detail: ☑ Assembly: ☐	14. Full FAI: ☑ Partial FAI: ☐ Baseline Part Number (Including Revision Level)		
	Reason for Full/Partial FAI: Lapse in production (>24 mo.) Reason for Full/Partial FAI (Comments):		AOG ☐ FAA Approved ☐
INDEX of part numbers or sub-assembly numbers required to make the assembly noted above.			
15. Part Number	16. Part Name	17. Part Type	Supplier / 18. FAIR Identifier
FAIR #1047 Contains No Parts			
Customer INCORA		Program UK DUAL USE	
From Division - To Division - Pattonair - EU Hub			
19. Does FAIR Contain a Documented Nonconformance(s): Yes ○ No ◉			
20. FAIR Verified By Reese Hooper Digitally signed using Net-Inspect		21. Date December 10, 2025 7:49:56 PM UTC+00:00	
22. FAIR Reviewed/Approved By Ryan Hooper Digitally signed using Net-Inspect		23. Date December 10, 2025 7:51:05 PM UTC+00:00	
24. Customer Approval Anna Boreboska Digitally signed using Net-Inspect		25. Date December 17, 2025 6:22:24 AM UTC+00:00	
26. Comments			

PURCHASE ORDER

GOODRICH ACTUATION SYSTEMS LIMITED
Purchase Order : 11827326

Created: 25.05.2022

Collins Aerospace

To the extent Supplier is subject to NIST SP 800-171 security requirements in accordance with DFARS 252.204-7012, Supplier represents that it has (1) completed within the last 3 years and will maintain at least a current basic NIST SP 800-171 DOD Assessment for all covered contractor information systems related to its business with Collins Aerospace that are not part of an information technology service or system operated on behalf of the Government and (2) submitted or will submit to the Government for posting to the USG's Supplier Performance Risk System (SPRS), the information required by paragraph (d) of DFARS 252.204-7020 prior to accepting this order from Collins Aerospace.
A CERTIFICATE OF CONFORMITY IS REQUIRED WITH THE GOODS. Work to be undertaken in accordance with the requirements of ASQR-01 & 981-151-001.
Special process operations to be controlled per 981-151-005.
Methods of electrical (EDM/ECM) machining must be controlled in accordance with procedure 917-020-005, 917-020-006 and 917-020-007.

Item	Material Description	Quantity Price	Unit of Measure Per Unit
00010	91213164 SOCKET HEAD SCREW 4 UNC DWG REV OR DIR: 2 ISSUE:	 Net Value:	each 1

Vendor Promise Date: 01.08.2022
Vendor Material No:
Item Last Changed On: 25.05.2022
Target QM:Quality System Approval Required
Inspection: FAI Required - Full/New Part Introduction
Contract:COMMERCIAL
Tax Code: IDT input driver tax code
Export Control #: EC188
Internal Order:
NAICS: 000000 ADVISORY BOARD UNDETERMINED

Required Measured Date:01.08.2022
Condition:
DDAS Rating:
Leadtime:
Tariff Number: 7318155290

Doc type	Document number	Document part	Doc version	Document description
Z18	91213164	000	3	SOCKET HEAD SCREW 4 UNC

UID Relevant: No

Item	Material Description	Target Quantity Price	Unit of Measure Per Unit
00010	CH1541-0095 BOSH - SPLIT DWG REV OR DIR: A ISSUE:	108,695 [redacted] Net Value: [redacted]	each [redacted]

Vendor Promise Date:
Vendor Material No:
Item Last Changed On: 04.11.2025
Target QM:Quality System Approval Required
Inspection: Subject to Inspection at UTAS
Contract:COMMERCIAL
Tax Code: IDT input driver tax code
Export Control #: PL9009a.2.a
Internal Order:

Required Measured Date:
Condition:
DDAS Rating:
Leadtime: [redacted]
Tariff Number: 3926909758

Doc type	Document number	Document part	Doc version	Document description
Z18	CH1541-0095	000	A	BOSH - SPLIT

UID Relevant: NO

Valid from 10.08.2015 to 31.12.2015
Unit Price [redacted] GBP 1 EA

Valid from 01.01.2016 to 31.12.2019
Unit Price [redacted] GBP 1 EA

Print Date: 04.11.2025 Time: 13:05:01

Python Automation Code

```python
def paste_header(FAIRpdf, part_number, rev, po_number, item_number, vendor_code, ref_num):
    """
    Paste a header in the top-right corner with a rectangle border.
    Text is rendered *inside* the border and placed a little lower.
    The box auto-expands if text doesn't fit.
    """

    doc = fitz.open(FAIRpdf)
    page = doc[0]

    # Header text (use & instead of &amp;)
    header_text = (
        "PATTONAIR LTD\n"
        f"Vendor #: {vendor_code}\n"
        f"PO: {po_number} & Item {item_number}\n"
        f"PN: {part_number} rev: {rev}\n"
        f"Unique FAIR Id: {ref_num}"
    )

    # ---- Layout parameters ----
    fontsize = 9
    fontname = "helv"          # if this yields no text, try "Helvetica" or remove fontname
    line_spacing = 1.2         # leading multiplier
    pad_x = 5                  # left/right padding inside border
    pad_top = 6                # top padding (higher so first line clears the border)
    pad_bottom = 6
```

```python
def signature(sig):
    """
    Function puts signature in Box 23 and date in Box 24
    and saves ONLY the first page.
    """

    # Get today's date
    today = date.today()
    formatted_date = today.strftime("%B %d, %Y")

    # Open original PDF
    doc = fitz.open("Form1_signed.pdf")
    page = doc[0]

    # Text labels to search for
    box23_text = "24. Customer Approval"
    box24_text = "25. Date"

    box23_coords = None
    box24_coords = None

    # Search for Box 23
    box23_instances = page.search_for(box23_text)
    if box23_instances:
        rect = box23_instances[0]
        box23_coords = (rect.x1 + 10, rect.y1)
```

```python
def stamp():
    """
    Function adds:
      1) a stamp under the signature in Box 24 (bottom-left area) on page 1, and
      2) a second stamp under the header (top-right area) on page 1.
    Writes Form1_signed.pdf and removes the intermediate Form1_signed_first_page_only.pdf.
    """

    # --- Inputs ---
    src_path = "Form1_signed_first_page_only.pdf"
    stamp_path = "stamp.pdf"
    out_path = "Form1_signed.pdf"

    # --- Open PDFs ---
    doc = PdfReader(src_path)
    stamp_doc = PdfReader(stamp_path)
    writer = PdfWriter()

    base_page = doc.pages[0]
    stamp_page = stamp_doc.pages[0]

    # Page size (points)
    page_width = float(base_page.mediabox.width)
    page_height = float(base_page.mediabox.height)

    # --- 1) Existing: bottom-left stamp (under signature / Box 24) ---
    # You can tune these to align perfectly with your signature box.
    target_width1 = 200  # width of the stamp in points
    left_margin1 = 50    # distance from left edge
    bottom_margin1 = 35  # distance from bottom edge (increase to move upwards)
```

```python
from header_functions import list_files, extract_data, paste_header, signature, stamp
from order import redact_po

path = os.getcwd()

ref_num = int(input("Unique Reference Number from GEAC: "))
sig = input("Signature in box 23: ")


if __name__ == "__main__":
    try:
        # get files in folder
        POpdf, FAIRpdf = list_files(path)

        # # HEADER
        po_number, item_number, vendor_code, part_number, rev = extract_data(POpdf, FAIRpdf)
        paste_header(FAIRpdf, part_number, rev, po_number, item_number, vendor_code, ref_num)

        # SIGNATURE & STAMP
        signature(sig)
        stamp()
        print("\nFORM 1 HAS BEEN UPDATED WITH A HEADER AND SIGNATURE.")

        # REDACT PO
        redact_po(POpdf, fill=(0, 0, 0))
        print("\nPRICES AND LEADTIMES HAVE BEEN COVERED IN THE PO. PLEASE CHECK.")

    except Exception as e:
        print("Error:", e)
```

```python
# ------------------------------
# MAIN PIPELINE
# ------------------------------

def redact_po(POpdf, fill=(0, 0, 0)):

    doc = fitz.open(POpdf)

    summary_labels = [
        "Original Award Value:",
        "Net Value",
        "Leadtime",
        "Tax",
        "Total Amount",
        "Total Net Value",
        "Total Net Value With No Tax USD",
        "Total Net Value Including Tax GBP",
    ]

    for page in doc:
        words = page.get_text("words")
        words.sort(key=lambda w: (round(w[1], 2), w[0]))
```

AI-Driven Data Analysis & Workflow Automation – Proof of Concept

Python Llama 3 Ollama Pandas PostgreSQL SQLAlchemy Matplotlib

Business Problem

Exploring ways to make structured data easier to query and analyze without requiring users to write SQL queries or manually create visualizations.

Solution

Developed a proof of concept integrating a locally hosted Llama 3 model with Excel data processing and a PostgreSQL database, enabling natural-language queries, AI-generated responses, and dynamic data visualizations.

Technical Implementation

- Integrated a locally hosted Llama 3 model through Ollama.
- Processed structured data from Excel using Python and Pandas.
- Connected to PostgreSQL and managed database interactions using SQLAlchemy.
- Generated analytical responses and dynamic charts using Matplotlib.

Business Impact

Demonstrated the potential to simplify data exploration, reduce manual analysis steps, and make insights more accessible through natural-language interaction.

Python Code

```
65    # PostgreSQL connection
66    engine = create_engine(
67        "postgresql+psycopg2://postgres:Clio42018@localhost/procurement_ai"
68    )
69
70    # question = input("Ask Procurement AI: ")
71    question = "count items"
72
73    # Step 1: Generate SQL
74    schema_query = """
75    SELECT
76        table_name,
77        column_name
78    FROM information_schema.columns
79    WHERE table_schema='public'
80    ORDER BY table_name, ordinal_position
81    """
82
83    schema_df = pd.read_sql(schema_query, engine)
```

```
126   sql_prompt = f"""
127   You are a PostgreSQL expert.
128
129   AVAILABLE TABLES AND COLUMNS:
130
131   {schema_text}
132
133   CRITICAL:
134
135   The database contains ONLY these tables:
136   - demand
137   - supplier_costs
138
139   IMPORTANT:
140   - Use ONLY the tables listed above.
141   - Use ONLY the columns listed above.
142   - Never invent table names.
143   - Never invent column names.
144   - If a table is not listed, do not use it.
145   - Return SQL only.
146   - Only discuss information present in the results.
147   - Do not make assumptions.
148   - If the result set is empty, explain that no records matched the criteria.
149   - Keep the answer concise.
```

```
    Question:
    {question}

    SQL:
    SELECT
        s.supplier_name,
        SUM(s.cost * d.annual_demand) AS annual_spend
    FROM supplier_costs s
    JOIN demand d
        ON s.part_number = d.part_number
    GROUP BY s.supplier_name
    ORDER BY annual_spend DESC;

    Now answer:

    {question}

    Rules:
    - Use PostgreSQL syntax only.
    - Do not use ROW_NUMBER() in WHERE clauses.
    - For top N results use ORDER BY ... LIMIT N.
    - Return SQL only.
    """

    response = ollama.chat(
        model="llama3",
        messages=[{"role": "user", "content": sql_prompt}]
    )

    sql = response["message"]["content"]
```

```
import matplotlib.pyplot as plt

if df.empty:
    print("No chart: query returned no data.")

else:

    numeric_cols = df.select_dtypes(include="number").columns.tolist()

    text_cols = [
        c for c in df.columns
        if pd.api.types.is_string_dtype(df[c])
    ]

    # print("Available columns:", df.columns.tolist())

    # Single KPI result (COUNT, SUM, AVG)
    if len(df) == 1 and len(numeric_cols) == 1:

        value = float(df.iloc[0][numeric_cols[0]])

        plt.figure(figsize=(5,5))
        plt.bar(["Result"], [value])
        plt.title(numeric_cols[0].replace("_", " ").title())
        plt.tight_layout()
        plt.show()
```

LET’S CONNECT

Python Automation & Process Optimization Engineer

Building automation that simplifies workflows, connects systems, and turns data into actionable insights.

Email: karolina.lecznar@yahoo.com

LinkedIn: Karolina Lęcznar

Phone: (+48) 796 960 199